

Pair-Sum State Collapse in Rainbow

Chosen-Prefix Collisions Against the Full Hash at Every Digest Width

Cris Stringfellow
DOSAYGO Corporation
research@dosaygo.com

AI-assisted research disclosure. The analysis reported here was conducted with substantial assistance from Anthropic’s Claude (Opus 5) for hypothesis refinement, derivation, implementation, and experiment design. The human author selected the target, reviewed the evidence, and takes responsibility for the claims and the manuscript. All computational evidence is published as source, witnesses, and machine-readable artifacts; every collision is replayed against the unmodified published implementation.

Abstract

Rainbow is a fast general-purpose hash with 64-, 128-, and 256-bit digests, documented by its author as non-cryptographic. We show that its 256-bit internal state carries only 128 bits of collision-relevant information, and that those 128 bits decompose into two independently attackable 64-bit halves.

The mechanism is an invariant of the message schedule. Rainbow injects each 16-byte block as $h_0 \text{ --} x$, $h_1 \text{ +=} x$, $h_2 \text{ +=} y$, $h_3 \text{ --} y$, so injection preserves the two *pair sums* $\sigma_1 = h_0 + h_1$ and $\sigma_2 = h_2 + h_3$ exactly while moving the state freely inside the coset they define. We prove that two states at the same block index can be driven to the *identical* state by one further message block, with zero search, if and only if their pair sums agree. Because mixA acts independently on (h_0, h_1) and (h_2, h_3) , the two sums are refreshed by two independent 64-bit functions, so equalizing them costs two 64-bit birthday searches rather than one 2^{128} search.

This yields collisions against *full, unreduced* production Rainbow. A same-seed, equal-length pair of 32-byte messages was found in $2^{32.28}$ evaluations and 6.6 seconds on one 8-core machine. A chosen-prefix collision between two semantically opposed 96-byte messages was found in $2^{34.80}$ evaluations and 50 seconds. Because the merge is a full internal-state collision occurring before finalization, one message pair collides at 64, 128, and 256 bits simultaneously.

The 64-bit variant is unaffected in the sense that 2^{32} is its generic bound. The substantive finding concerns the wider digests: rainbow-128 and rainbow-256 provide no more collision resistance than rainbow-64, a shortfall of 2^{32} and 2^{96} respectively against their generic bounds. We claim no break of a stated security property, since Rainbow claims none; we claim that the advertised output widths do not deliver proportional collision resistance, and that this follows from a structural property rather than from search. All results are reproduced by four independent implementations, including the shipped `rainsum` command-line tool.

Keywords: Rainbow; hash functions; collision attacks; chosen-prefix collisions; non-cryptographic hashing; state collapse; AI-assisted cryptanalysis.

1 Introduction

Rainbow is a word-oriented hash with a 256-bit internal state, 16-byte input blocks, a 64-bit seed, and 64-, 128-, or 256-bit digests. Its documentation is explicit that it is a general-purpose, non-cryptographic hash and is “not designed as cryptographic” [2]. That disclaimer is important and we take it seriously: nothing in this paper contradicts a claim Rainbow makes.

It is worth stating plainly why a collision attack on such a function is of interest at all. Collisions in fast non-cryptographic hashes are usually easy, and often far easier than the attack presented here: CRC32 is linear and collides by construction, and published collision constructions exist for xxHash, MurmurHash3, and CityHash at negligible cost. A 2^{32} search against a 64-bit digest is simply the generic birthday bound and is not a cryptanalytic result.

The result here is of a different kind, and rests on two observations.

First, the attack does not target a digest; it targets the internal state, and it succeeds against the *full* function. Rainbow maintains 256 bits of state, but we show that only 128 of those bits can ever distinguish two states from the standpoint of an attacker who controls one more block, because message injection preserves two 64-bit quantities and lets the attacker place the state anywhere else in their coset. The state does not merely mix poorly; it is, for collision purposes, 128 bits wide.

Second, those 128 bits split. The mixA mixer never moves information between (h_0, h_1) and (h_2, h_3) , so the two pair sums are refreshed by two separate 64-bit maps. Divide and conquer applies, and the cost of merging two arbitrary states is two 64-bit searches – roughly 2^{33} – instead of the 2^{128} that a 256-bit state should demand.

The consequence is that the digest width is decorative with respect to collisions. The same pair of messages collides at 64, 128, and 256 bits, because the states are already equal before finalization begins. Rainbow-256 costs no more to collide than rainbow-64.

Contributions.

- A proof that two Rainbow states merge under one message block, with zero search, exactly when their two pair sums agree (Theorem 1), validated on 1000/1000 random state pairs.
- A reduction of full-state collision search to two independent 64-bit problems (Section 5).
- A same-seed collision on full Rainbow in $2^{32.28}$ evaluations, with witnesses colliding at all three widths (Section 6).
- A chosen-prefix collision in $2^{34.80}$ evaluations between two messages with opposed meanings (Section 7).
- Independent verification by four implementations, including the shipped CLI (Section 9).

2 Background: Rainbow v3.7.1

We describe the function as implemented in `src/rainbow.cpp` at version 3.7.1, which is the authority for every claim below. All arithmetic is in $\mathbb{Z}/2^{64}\mathbb{Z}$; $\text{ROTR}(x, n)$ is a 64-bit rotation.

2.1 Constants and mixers

Rainbow uses eight odd 64-bit constants P, Q, R, S, T, U, V, W , with $P = 2^{64} - 59$. The state is $h = (h_0, h_1, h_2, h_3)$. The primary mixer `mixA` is

$$a \leftarrow \text{ROTR}(h_0 \cdot P, 23) \cdot Q, \quad (1)$$

$$b \leftarrow \text{ROTR}((h_1 \oplus a) \cdot R, 29) \cdot S, \quad (2)$$

$$c \leftarrow \text{ROTR}(h_2 \cdot T, 31) \cdot U, \quad (3)$$

$$d \leftarrow \text{ROTR}((h_3 \oplus c) \cdot V, 37) \cdot W, \quad (4)$$

with $h \leftarrow (a, b, c, d)$. The secondary mixer `mixB`, which takes the seed as an additional input, rewrites only h_1 and h_2 and is followed by a rotation of the four words.

Remark 1 (`mixA` is pairwise separable). Equations (1)–(2) read only h_0, h_1 and write only a, b ; equations (3)–(4) read only h_2, h_3 and write only c, d . `mixA` therefore acts as two independent maps on $\mathbb{Z}/2^{64}\mathbb{Z}^2$. This is the property the attack exploits, and it is visible directly in the source.

2.2 Message schedule

For each complete 16-byte block, Rainbow parses two little-endian words x, y and injects

$$h_0 \text{ --} x, \quad h_1 \text{ +=} x, \quad h_2 \text{ +=} y, \quad h_3 \text{ --} y. \quad (5)$$

Blocks alternate mixers: the first complete block is followed by `mixA`, the second by `mixB` and a rotation, and so on. Writing blocks 1-indexed, block n is followed by `mixA` exactly when n is odd.

The state is keyed by the total message length: $h_i = \text{seed} + \text{len} + \{1, 2, 3, 5\}$ for $i = 0, 1, 2, 3$. Two messages therefore share an initial state if and only if they share a seed and a length, and every collision below is between equal-length messages.

After the block loop, Rainbow applies `mixB`, absorbs the 0–15 tail bytes at length-determined lanes, then applies `mixA`, `mixB`, `mixA` and emits digest words.

Remark 2 (The output is itself a pair sum). Each 64-bit digest word is computed as $\mathbf{g} = 0$; $\mathbf{g} \text{ --} \mathbf{h}[2]$; $\mathbf{g} \text{ --} \mathbf{h}[3]$; that is $-\sigma_2$ for the current state. The finalization reads out precisely the quantity that the message schedule preserves. We do not need this observation for the attack, which achieves full-state equality, but it indicates that the pair sums are load-bearing at both ends of the construction.

3 Threat Model, Notation, and Scope

We work in the fixed public-seed setting with $\text{seed} = 0$; nothing depends on that choice, and the attack is stated for arbitrary known seeds. The attacker chooses complete messages and knows the seed. There is no key and no secret, so no PRF or MAC claim is in scope.

Write $\sigma_1(h) = h_0 + h_1$ and $\sigma_2(h) = h_2 + h_3$ for the two *pair sums*. We say two states are *mergeable* if some choice of one further message block for each yields identical states.

We attack the full, unreduced function: every mixer, the complete schedule, the finalization, and all three digest widths. No component is weakened, removed, or reduced in round count, and the attack code does not link against production sources, so it cannot perturb them.

4 The Pair-Sum Invariant

Lemma 1 (Injection preserves both pair sums). *Let h' be the result of injecting (x, y) into h by (5). Then $\sigma_1(h') = \sigma_1(h)$ and $\sigma_2(h') = \sigma_2(h)$, for every $x, y \in \mathbb{Z}/2^{64}\mathbb{Z}$.*

Proof. $\sigma_1(h') = (h_0 - x) + (h_1 + x) = h_0 + h_1$ and $\sigma_2(h') = (h_2 + y) + (h_3 - y) = h_2 + h_3$, all in $\mathbb{Z}/2^{64}\mathbb{Z}$. $\square$

The message words move the state along the two cosets and cannot leave them. The attacker's freedom inside a coset is total: x and y range over all of $\mathbb{Z}/2^{64}\mathbb{Z}$. This makes the pair sums an exact invariant of the schedule and, as the next theorem shows, a complete one.

Theorem 1 (Pair-sum equivalence). *Let S and T be Rainbow states at the same block index. There exist message words (x, y) for S and (x', y') for T whose injection yields identical states if and only if*

$$\sigma_1(S) = \sigma_1(T) \quad \text{and} \quad \sigma_2(S) = \sigma_2(T).$$

When the condition holds, the solution set has size 2^{128} : x and y may be chosen freely, and then

$$x' = x + (t_0 - s_0), \quad y' = y + (t_3 - s_3).$$

Proof. ($\Leftarrow$) Injection yields $(s_0 - x, s_1 + x, s_2 + y, s_3 - y)$ and $(t_0 - x', t_1 + x', t_2 + y', t_3 - y')$. Matching word 0 requires $x' = x + (t_0 - s_0)$; matching word 1 requires $x' = x + (s_1 - t_1)$. The two are consistent precisely when $t_0 - s_0 = s_1 - t_1$, i.e. $s_0 + s_1 = t_0 + t_1$. Symmetrically, word 3 requires $y' = y + (t_3 - s_3)$ and word 2 requires $y' = y + (s_2 - t_2)$, consistent precisely when $s_2 + s_3 = t_2 + t_3$. Under the hypothesis both hold, and x, y remain free.

($\Rightarrow$) If the post-injection states are identical they have equal pair sums, and by Lemma 1 injection did not change either state's pair sums, so S and T had equal pair sums already. $\square$

Corollary 1 (Effective state width). *For collision finding, the collision-relevant content of a Rainbow state is the 128-bit pair (σ_1, σ_2) , not the 256-bit state. Two states agreeing on it are one attacker-chosen block away from being the same state, at no search cost.*

Note the direction of the “if and only if”: the theorem is not merely a sufficient trick but a characterization. The residual 128 bits of state beyond the pair sums are attacker-erasable.

Experimental Result 1 (Theorem validation). Over 1000 random trials, state pairs were constructed with independently random words subject only to matching pair sums, and merged by the closed form above. All 1000 produced bitwise-identical states after injection, and identical states after each of mixA and mixB with rotation. Implemented in `research/rainbow/chosen_prefix_collision.cpp` as `theorem_selftest`.

5 Independence of the Two Halves

Corollary 1 reduces the merging problem to matching 128 bits. Remark 1 reduces it further, to two independent 64-bit problems.

Definition 1 (Half-sum maps). For a state S at a mixA boundary, define

$$\begin{aligned} A_S(x) &= \text{ROTR}((s_0 - x) \cdot P, 23) \cdot Q, & B_S(x) &= \text{ROTR}(((s_1 + x) \oplus A_S(x)) \cdot R, 29) \cdot S, \\ C_S(y) &= \text{ROTR}((s_2 + y) \cdot T, 31) \cdot U, & D_S(y) &= \text{ROTR}(((s_3 - y) \oplus C_S(y)) \cdot V, 37) \cdot W, \end{aligned}$$

and set $H_S(x) = A_S(x) + B_S(x)$ and $K_S(y) = C_S(y) + D_S(y)$.

H_S is the value of σ_1 after injecting x and applying mixA; K_S is the value of σ_2 after injecting y and applying mixA. By Remark 1, H_S depends only on x and K_S only on y .

Proposition 1 (Two-claw merging). *Let S, T be states at a mixA boundary. If $H_S(x) = H_T(x')$ and $K_S(y) = K_T(y')$, then injecting (x, y) into S and (x', y') into T , applying mixA to each, and injecting the bridge words*

$$p = -B_S(x), \quad q = -C_S(y) \quad \text{and} \quad p' = -B_T(x'), \quad q' = -C_T(y')$$

produces identical states, namely $(H, 0, 0, K)$.

Proof. After mixA the first pair is (A, B) and the second is (C, D) . The bridge injection sends $(A, B) \mapsto (A - p, B + p)$ and $(C, D) \mapsto (C + q, D - q)$. With $p = -B$ the first pair becomes $(A + B, 0) = (H, 0)$; with $q = -C$ the second becomes $(0, C + D) = (0, K)$. The hypotheses make the two H values and the two K values equal, so the four words agree. (Any other common point of the cosets would serve equally; this one is convenient because it is canonical.) $\square$

Since H and K behave as random 64-bit maps, each condition is met after roughly 2^{32} evaluations by birthday search, for a total near 2^{33} rather than 2^{128} .

Remark 3 (H is not a permutation). A_S is a bijection of x , and $t \mapsto \text{ROTR}(tR, 29)S$ is a bijection, so one might hope $H_S = A_S + B_S$ is a permutation, which would make the single-search variant of Section 6 impossible. It is not: the collision exhibited in Result 2 is an explicit pair $x_a \neq x_b$ with $H(x_a) = H(x_b)$, and the search behaved throughout like a search on a random map. The XOR in (2) sits between two additive structures and destroys any such algebraic balance.

6 Same-Seed Collision on Full Rainbow

When the two messages share a seed and a length, they share an initial state, and one search suffices.

Construction. Fix message length 32 bytes (two blocks) and seed 0, giving $S = (33, 34, 35, 37)$. Let both messages use the same second word $y = 0$ in block 1, so the second pair evolves identically and K agrees automatically. Search for $x_a \neq x_b$ with

$$H(x_a) = H(x_b).$$

Emit

$$m_i = \text{LE}_{64}(x_i) \parallel \text{LE}_{64}(0) \parallel \text{LE}_{64}(-B(x_i)) \parallel \text{LE}_{64}(0).$$

Block 2's first word cancels B , so both states become $(H, 0, C, D)$ with C, D common. The states are equal before block 2's mixer, hence for the rest of the computation.

Search. We use a van Oorschot–Wiener parallel distinguished-point rho on H [4]: walks iterate $x \mapsto H(x)$ until the low d bits vanish, endpoints are stored, and a repeated endpoint is re-walked to expose the colliding pair. Memory is negligible.

Experimental Result 2 (Same-seed collision). With $d = 20$ and 8 threads, a collision was found in 5,206,393,642 evaluations ($2^{32.28}$), 4920 walks, one endpoint merge, 6.63 s wall, 785.7 Meval/s.

x_a	0xf2361ca7274d362f
x_b	0x17f81881806268c1
H	0x26d62ded70d34542 (both)
m_a	2f364d27a71c36f20000000000000000dbc4cde348f671ec00000000000000000
m_b	c16862808118f8170000000000000000ebc98b7034e669320000000000000000
rainbow-64	b013b427e381f006
rainbow-128	b013b427e381f006c89c06357350be4b
rainbow-256	b013b427e381f006c89c06357350be4be142f3bdc749f11c58acc1b99a4d91e3

The merged state after block-2 injection is (26d62ded70d34542, 0, 1b0560be4a530f25, b8edca491361c74b) for both messages.

7 Chosen-Prefix Collision

The same-seed attack requires the messages to agree before the collision block. Theorem 1 and Proposition 1 remove that restriction entirely: *any* two states can be merged, so the prefixes may be chosen arbitrarily and independently.

Construction. Let $\text{pre}_a \neq \text{pre}_b$ be prefixes of equal length, padded to a multiple of 32 bytes so that the next block is a mixA block. Absorb them to obtain S and T . Solve two claws,

$$H_S(x_a) = H_T(x_b), \quad K_S(y_a) = K_T(y_b),$$

then append to each message its claw block $\text{LE}_{64}(x) \parallel \text{LE}_{64}(y)$ and its bridge block $\text{LE}_{64}(-B) \parallel \text{LE}_{64}(-C)$. By Proposition 1 the states coincide at the bridge injection.

A claw between two functions is obtained as an ordinary collision of the flavored combination $F(w) = H_{\beta(w)}(w)$, where $\beta(w)$ is a cheap bit derived from w ; collisions with differing flavor are claws. Roughly half the collisions are same-flavor and discarded, so a claw costs about twice a plain collision. This accounts for the observed $2^{33.9}$ per claw against the 2^{32} of Result 2; a dedicated two-sided search would reduce it.

Experimental Result 3 (Chosen-prefix collision). Two 96-byte messages with opposed meanings and identical digests at all three widths. Prefixes were space-padded to 64 bytes; the padding is part of the chosen prefix.

prefix <i>a</i>	From: <code>alice@example.com</code> Amount: USD 10.00
prefix <i>b</i>	From: <code>mallory@example.net</code> Amount: USD 1000000.00
claw <i>H</i>	16,190,842,307 evaluations ($2^{33.91}$), 27.1 s
claw <i>K</i>	13,673,595,949 evaluations ($2^{33.67}$), 23.0 s
total	29,864,438,256 evaluations ($2^{34.80}$), 50.0 s
merged state	(0bcacc509d3457d6, 0, 0, 7f92191466aadf76)
rainbow-64	913ac822c5f4c8fe
rainbow-128	913ac822c5f4c8fe40cb08c0af232d8a
rainbow-256	913ac822c5f4c8fe40cb08c0af232d8ab58e007cc27e827269d591fd423682db

7.1 Corollaries

Because a merge produces genuinely identical internal states, the standard generic consequences follow immediately and at the same per-merge cost.

- **Multicollisions.** Chaining t successive same-state collision gadgets in the manner of Joux [5] yields 2^t colliding messages for $t \cdot 2^{32}$ work. A 2^{64} -multicollision costs about 2^{38} .
- **Common-suffix forgery.** Any common suffix may be appended to a merged pair, at every digest width, since the states are equal.
- **Herding and expandable messages.** The cheap merge of arbitrary states is the primitive underlying diamond-structure herding [7] and long-message second-preimage techniques [6]; we did not implement these.

We emphasize that only the first two items were executed; the third is stated as an expected consequence, not a demonstrated result.

8 Why the Digest Width Does Not Help

Rainbow’s finalization is a deterministic function of the internal state. The attack achieves equality of that state before finalization begins, so every subsequent operation, and hence every digest word at every width, agrees. This is visible in the witnesses: the 64-bit digest is a prefix of the 128-bit digest, which is a prefix of the 256-bit digest, for both messages.

Variant	Generic collision cost	Cost here	Shortfall
rainbow-64	2^{32}	$2^{32.3}$	none (generic)
rainbow-128	2^{64}	$2^{32.3}$	$\approx 2^{32}$
rainbow-256	2^{128}	$2^{32.3}$	$\approx 2^{96}$

The honest reading is narrow and, we think, still worth stating. Rainbow-64 is not weakened: 2^{32} is what a 64-bit digest costs. What fails is the implicit proposition that selecting a wider digest buys proportionally more collision resistance. It buys none. A user choosing rainbow-256 over rainbow-64 for collision avoidance in a large data set receives no benefit against an adversary, and the cost of demonstrating this is seconds on a laptop.

This is a property of the construction, not of the search: a 256-bit state whose message schedule preserves two 64-bit invariants, and whose mixer never crosses the halves, cannot offer more than 64-bit-per-half resistance no matter how many finalization rounds follow.

9 Experimental Results and Independent Verification

Every witness was checked by four implementations that do not share code:

1. **The attack’s own model.** `collision_pair_sum.cpp` and `chosen_prefix_collision.cpp` each transcribe Rainbow independently and self-check their witnesses.
2. **An independent Python implementation.** `verify_pair_sum.py` re-implements Rainbow with big integers, re-derives the pair sums from the message bytes alone, prints a checkpoint-by-checkpoint comparison, and confirms the digests. It reports the merge point: block-2 injection for Result 2, block-6 injection for Result 3.
3. **The unmodified production source.** `replay_production.cpp` includes `src/rainbow.cpp` verbatim and hashes the serialized bytes, never in-memory words.
4. **The shipped command-line tool.** `rainsum -a bow -s {64,128,256}` on the message files reproduces the same digests.

All four agree on both witnesses at all three widths. Messages are carried as hex and replayed from bytes, so no endianness assumption is shared between the search and the verification.

10 Interpretation and Recommendations

Rainbow is documented as non-cryptographic, and this paper does not assert that it fails a claim it makes. Two narrower statements are supported by the evidence.

First, the 128-bit and 256-bit variants should not be described or selected as offering collision resistance commensurate with their width. Their collision cost is that of the 64-bit variant.

Second, the cause is structural and admits a targeted remedy. Any one of the following breaks the invariant chain, and we suggest them as directions rather than as a validated fix:

- inject with coefficients that do not cancel, so that σ_1, σ_2 are not preserved (for example, injecting x and $\text{ROTR}(x, k)$ rather than x and $-x$);
- make mixA cross the halves, so that the two 64-bit problems cannot be solved independently;
- avoid reading the digest out as $-\sigma_2$ (Remark 2).

We have not analyzed any modified variant, and offer no assurance that these suffice.

Finally, the general lesson is about state accounting. Rainbow’s state is 256 bits, but its *collision-relevant* state is 128 bits, and its *independently attackable* state is 64 bits per half. When a message schedule has an exact invariant that the attacker can also move freely within, the invariant – not the register file – is the true state.

11 Reproducibility

All sources, witnesses, and machine-readable results are in `research/rainbow/`. From the repository root:

```
# Build (any C++20 compiler; -march=native optional)
clang++ -std=c++20 -O3 -march=native -o research/rainbow/bin/collision_pair_sum \
    research/rainbow/collision_pair_sum.cpp
clang++ -std=c++20 -O3 -march=native -o research/rainbow/bin/chosen_prefix_collision \
    research/rainbow/chosen_prefix_collision.cpp
clang++ -std=c++20 -O3 -march=native -o research/rainbow/bin/replay_production \
    research/rainbow/replay_production.cpp

# Same-seed collision (about 7 s on 8 cores)
./research/rainbow/bin/collision_pair_sum --seed=0 --dp-bits=20 --threads=8

# Chosen-prefix collision (about 50 s on 8 cores)
./research/rainbow/bin/chosen_prefix_collision --dp-bits=20 --threads=8

# Independent verification and production replay
python3 research/rainbow/verify_pair_sum.py research/rainbow/collision-result.json
python3 research/rainbow/verify_pair_sum.py research/rainbow/chosen-prefix-result.json
./research/rainbow/bin/replay_production <hex_a> <hex_b> 0
```

Searches are randomized and will return different witnesses on each run; `-rng-seed` fixes the stream. The published witnesses are recorded in `collision-result.json` and `chosen-prefix-result.json` together with full search accounting.

12 Conclusion

Rainbow’s message schedule preserves two 64-bit pair sums that the attacker can otherwise move at will, and its primary mixer never crosses the two halves of the state. Together these reduce a nominally 256-bit collision problem to two independent 64-bit searches. We proved that equal pair sums are necessary and sufficient for two states to merge under a single block with no search, and used the result to produce collisions against the full function: a same-seed pair in $2^{32.28}$ evaluations and a chosen-prefix pair in $2^{34.80}$, each colliding simultaneously at 64, 128, and 256 bits, each confirmed against the unmodified implementation.

Rainbow claims no cryptographic security, and none is refuted. What is refuted is the value of its wider digests against collisions: they cost no more to break than the narrowest one, and the reason is structural rather than computational.

References

- [1] DOSAYGO Corporation. *rain: Rainbow and Rainstorm hash functions*, `src/rainbow.cpp`, version 3.7.1. <https://github.com/DOSAYGO-STUDIO/rain>
- [2] DOSAYGO Corporation. *rain README*: “Rainbow ... General-purpose, non-crypto hash ... Not designed as cryptographic.”

- [3] C. Stringfellow. *A $2^{31.9}$ Collision Attack on Reduced Rainstorm-256: Programmed Collisions, Rebound Corridors, and the Full-Schedule Boundary*.
- [4] P. C. van Oorschot and M. J. Wiener. *Parallel Collision Search with Cryptanalytic Applications*. Journal of Cryptology, 12(1):1–28, 1999.
- [5] A. Joux. *Multicollisions in Iterated Hash Functions. Application to Cascaded Constructions*. CRYPTO 2004, LNCS 3152, pp. 306–316.
- [6] J. Kelsey and B. Schneier. *Second Preimages on n -Bit Hash Functions for Much Less than 2^n Work*. EUROCRYPT 2005, LNCS 3494, pp. 474–490.
- [7] J. Kelsey and T. Kohno. *Herdin Hash Functions and the Nostradamus Attack*. EUROCRYPT 2006, LNCS 4004, pp. 183–200.
- [8] M. Stevens, A. K. Lenstra, and B. de Weger. *Chosen-Prefix Collisions for MD5 and Colliding X.509 Certificates for Different Identities*. EUROCRYPT 2007, LNCS 4515, pp. 1–22.
- [9] R. C. Merkle. *One Way Hash Functions and DES*. CRYPTO 1989, LNCS 435, pp. 428–446.
- [10] Various contributors. *SMHasher3: Hash function quality and speed test suite*.